

Esercitazione

Gestione Input/Output

Architettura degli elaboratori

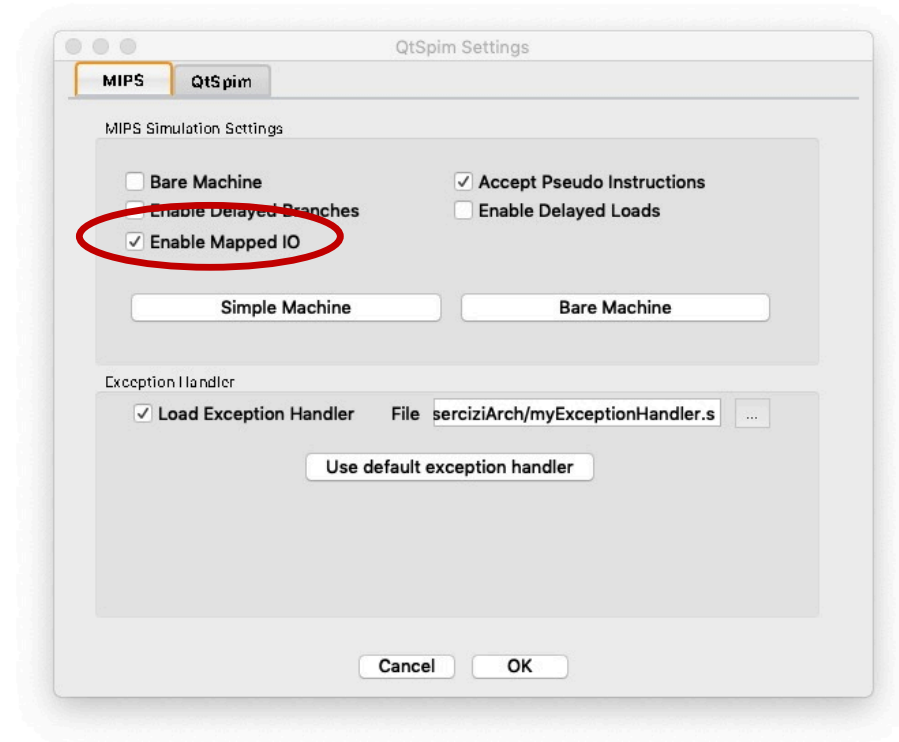
Tecniche di gestione I/O

- Controllo di programma – busy wait
- Gestione tramite interrupt
- Direct Memory Access (DMA)

I/O in SPIM

- Con SPIM sono realizzabili le sole tecniche di gestione I/O busy wait e con interrupt
- SPIM simula una periferica di I/O (terminal device) che permette a un programma MIPS in esecuzione di:
 - leggere i caratteri digitati a tastiera
 - scrivere caratteri sulla console (finestra di output)

Nelle impostazioni di SPIM deve essere abilitata l'opzione Enable Mapped I/O



SPIM terminal device

- Il terminale SPIM è composto da due dispositivi **distinti** e **indipendenti**:
 - **Receiver**: legge i caratteri ASCII dalla tastiera
 - **Transmitter**: visualizza caratteri ASCII sulla Console (finestra di output)
- Le unità Receiver e Transmitter sono completamente **indipendenti**:
 - un tasto premuto sulla tastiera non viene automaticamente mostrato sulla console
 - un programma deve esplicitamente fare “eco” di un carattere letto dal Receiver inviandolo al Transmitter per mostrarlo sulla console

Gestione SPIM terminal device

- Per il controllo del device di I/O da parte di un programma sono utilizzati **4 registri**
- I registri per il controllo del device di I/O sono **memory-mapped** ovvero ognuno dei 4 registri è in realtà un'area di memoria di 32 bit che si trova ad uno specifico indirizzo

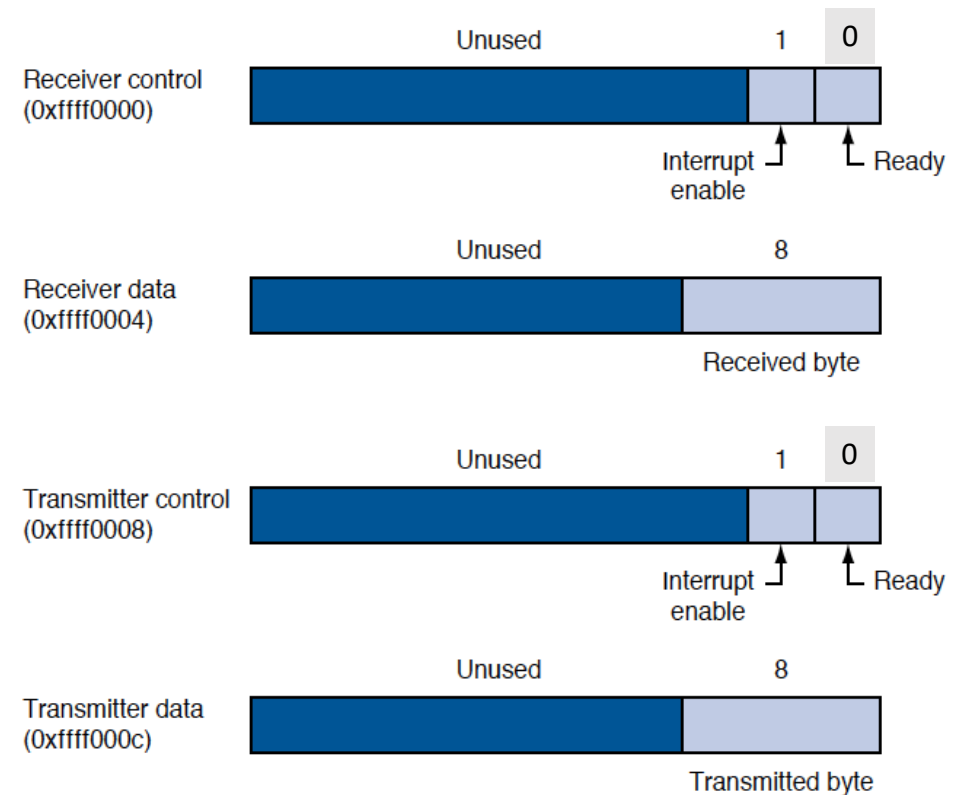
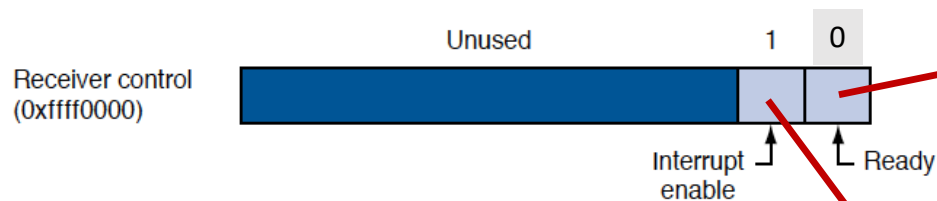


FIGURE A.8.1 The terminal is controlled by four device registers, each of which appears as a memory location at the given address. Only a few bits of these registers are actually used. The others always read as 0s and are ignored on writes.

Receiver Control register (address 0xffff0000)



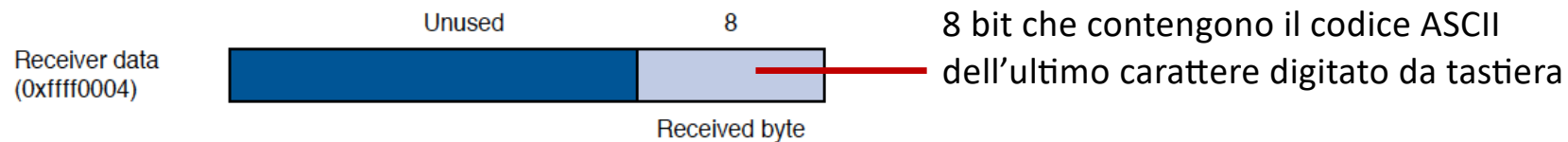
Bit 0 (keyboard ready bit):

- è in sola lettura (le scritture vengono ignorate)
- passa da 0 a 1 quando è digitato un carattere da tastiera
- resta a 1, fintanto che il carattere non è prelevato

Bit 1 (keyboard “interrupt enable” bit)

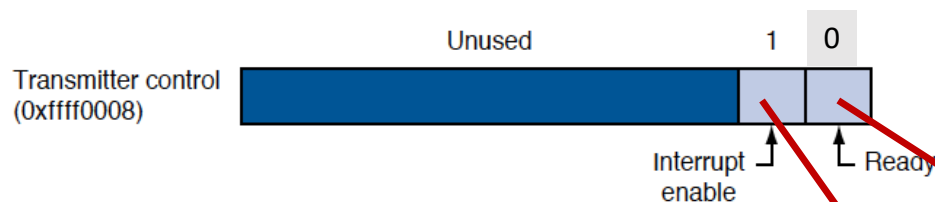
- può essere sia letto che scritto da un programma
- inizialmente è a 0
- se il programma lo mette a 1 (interrupt dal receiver abilitati), il terminale genera un interrupt ogni volta che è digitato un carattere (mettendo a 0 il codice eccezione e a 1 il bit corrispondente al Receiver nel **Cause register**)
- Perchè gli interrupt abbiano effetto sul comportamento del processore, **gli interrupt devono anche essere abilitati nello Status register**

Receiver Data register (at address 0xffff0004)



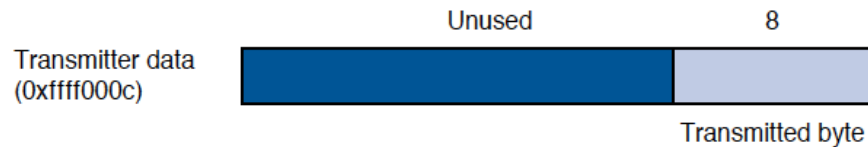
- “registro” read-only
- Cambia quando un nuovo carattere è digitato a tastiera → il codice ASCII del tasto premuto è scritto negli 8 bit meno significativi
- Il suo contenuto è valido soltanto se Ready bit del Receiver Control è 1 (non è definito se il Ready bit del Receiver Control è a 0)
- La sua lettura riporta a 0 il Ready bit del Receiver Control

Transmitter Control register (at address 0xffff0008)



- comportamento simile a Receiver Control register
- **Bit 0 (ready bit)**
 - read-only
 - se è a 1, il Transmitter è pronto per accettare un nuovo carattere
 - se è 0, il Trasmittente sta ancora scrivendo il carattere precedente
- **Bit 1 (interrupt enable bit)**
 - può essere letto o scritto da un programma
 - se è a 1, il terminale richiede un'interruzione all'hardware quando il trasmettitore è pronto (Ready bit è 1) per ricevere un nuovo carattere

Transmitter Data register (at address 0xffff000c)



- Quando un valore è scritto in questo registro,
 - gli 8 bit 0-7 (i.e., codifica ASCII di un carattere) sono trasmessi alla console
 - il Ready bit del registro Transmitter Control è messo a 0. Rimarrà a 0, finchè sarà trascorso un tempo sufficiente per trasmettere il carattere al terminale di output e poi il Ready bit tornerà ad essere 1
- Può essere scritto quando il Ready bit del registro Transmitter Control è a 1
- Se Ready bit è a 0 (il trasmitter non è pronto), i dati scritti nel Transmitter Data register sono ignorati

Interrupt

- Causati da dispositivi hardware esterni (periferiche I/O), collegati al processore attraverso delle linee di controllo di bus
- Quando le periferiche sono pronte ad effettuare operazioni attivano una linea di interruzione collegata alla CPU
- Quando viene attivata una linea di interruzione, ***se sono abilitate le interruzioni della CPU***, il normale flusso di esecuzione viene interrotto e la CPU passa a gestire l'interrupt (i.e. scrittura del registro Cause e esecuzione dell'exception handler)
- Registri del Co-processore 0 a supporto della gestione dell'I/O: **Cause** e **Status**

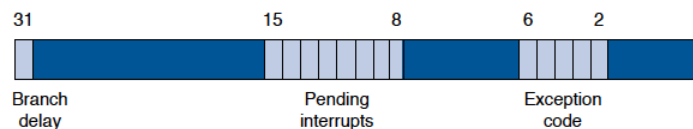


FIGURE A.7.2 The Cause register.

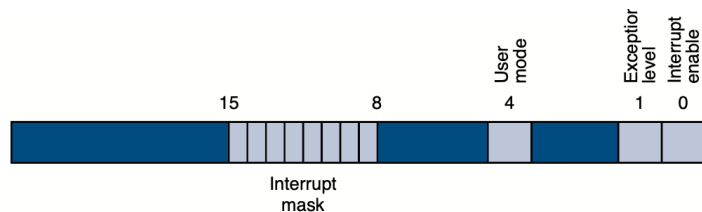


FIGURE A.7.1 The Status register.

Registri del Co-processor 0 disponibili in SPIM

Register name	Register number	Usage
BadVAddr	8	memory address at which an offending memory reference occurred
Count	9	timer
Compare	11	value compared against timer that causes interrupt when they match
Status	12	interrupt mask and enable bits
Cause	13	exception type and pending interrupt bits
EPC	14	address of instruction that caused exception
Config	16	configuration of machine

Status register del Coprocessore 0

- Interrupt enable (bit 0): interrupt abilitati (valore 1) o non abilitati (valore 0)
- Interrupt mask: ciascun bit consente l'abilitazione di una differente linea di interruzione collegata ad una o più periferiche
 - Se il bit è a 1, il corrispondente interrupt è abilitato e la periferica che solleva una eccezione può essere «percepita»
 - Se il bit è a 0, il corrispondente interrupt NON è abilitato e anche se la linea di interruzione è attivata dalla periferica, la CPU non genera l'eccezione
- Exception level: normalmente a 0. Impostato a 1 in caso di eccezione (per disabilitare gli interrupt mentre l'exception handler sta gestendo l'eccezione)

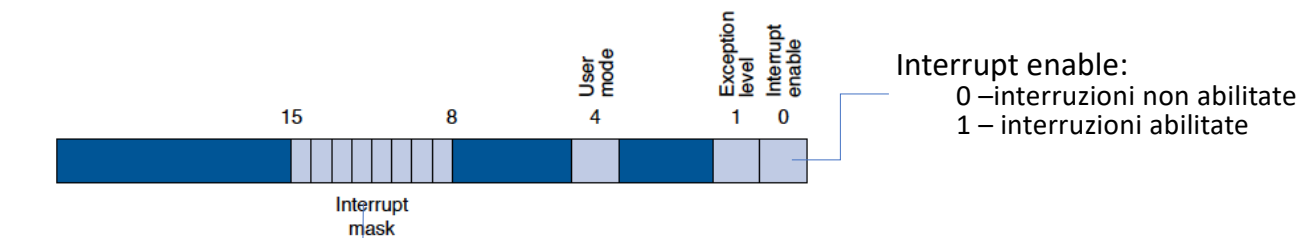


FIGURE A.7.1 The Status register.

Interrupt mask – **interrupt abilitati**

Un bit per ognuna delle 6 linee delle periferiche e
2 bit per interrupt a livello sw

Cause register del Coprocessore 0

- Quando viene sollevata una eccezione a seguito di un'interruzione, nel **Cause register** i 5 bit dell'**exception code** sono impostati a **0** e viene impostato a 1 il bit corrispondente alla periferica che ha inviato il segnale di interruzione (anche nel caso in cui il bit corrispondente nell'interrupt mask dello Status register fosse disabilitato)
- Il processore è interrotto se c'è un interrupt nel Cause register E il bit corrispondente nello Status register è abilitato (e le interruzioni sono abilitate)

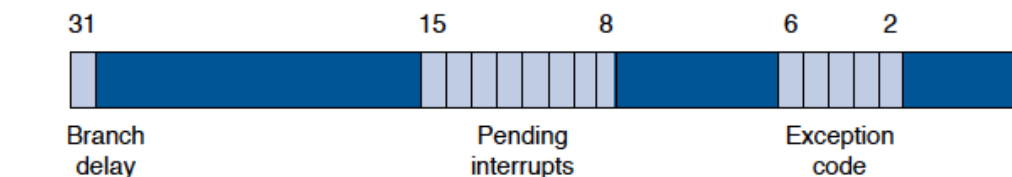


FIGURE A.7.2 The Cause register.

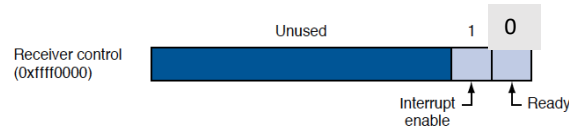
Esempio – diverse tecniche di gestione I/O

Realizzare un programma assembly che stampa tramite syscall, un carattere inserito da tastiera (utilizzando i registri Receiver Control e Receiver Data) attraverso le due tecniche:

- **busy wait**: gestione da parte del programma
- **interrupt**: gestione da parte dell'exception handler

Si faccia riferimento ai dettagli dell'implementazione SPIM (simulatore dell'architettura MIPS, descritto in Appendice A del manuale) per la gestione del device di I/O da parte di un programma → 4 registri memory-mapped (fig. A.8.1)

Esempio (busy wait)



.data

Messaggio:

.ascii "Il tasto premuto e': " #NB questa stringa non e'
#terminata con byte 0!

Carattere:

.asciiz "\n\n" # due a capo (il primo sara' sovrascritto)

Fine: .ascii "q" # carattere da premere per terminare

.text

main:

li \$t0, 0xFFFF0000 # Receiver Control
li \$t2, 0xFFFF0004 # Receiver Data
lb \$t4, Fine # carattere per finire

BusyWaitRead: # "busy wait"

lw \$t1, 0(\$t0) # Receiver Control Register
andi \$t9, \$t1, 0x1 # tiene solo il bit 0
beqz \$t9, BusyWaitRead # se=0, non e' arrivato nulla
se= 1, e' arrivato un carattere
lb \$t3, 0(\$t2) # legge byte dal Receiver Data
beq \$t3, \$t4, Uscita # se e' il tasto di fine, esce
sb \$t3, Carattere # salva il carattere letto in memoria
#sovrascrivendo il primo 'a capo'
li \$v0, 4 # stampa il messaggio con la syscall
la \$a0, Messaggio # il carattere e' incorporato !
syscall # stampa con syscall
j BusyWaitRead

Uscita:

li \$v0, 10 #terminazione
syscall

Esempio (con interrupt) – 1/3

Codice del programma che abilita gli interrupt e stampa una sequenza infinita di 0

.data

car0: .asciiz "0"

Messaggio: .ascii "Il tasto premuto e':"

Carattere: .asciiz "\n\n"

.text

main:

Abilita gli interrupt della CPU, in generale e da tastiera

mfc0 \$t0, \$12 # Leggi Status register (12)

ori \$t0, \$t0, 1 # Abilita in generale gli INT

ori \$t0, \$t0, 0x800 # Abilita gli INT da Tastiera

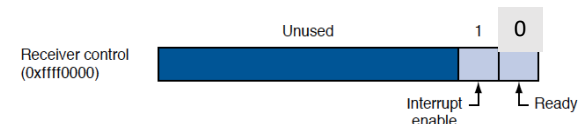
mtc0 \$t0, \$12 # Scrivi Status Register

Abilita gli interrupt della periferica I/O (Receiver)

lw \$t1, 0xFFFF0000 # Leggi Receiver Control

ori \$t1, \$t1, 2 # Abilita INT

sw \$t1, 0xFFFF0000 # Scrivi nuovo Receiver Control



Stampa in un ciclo infinito il carattere 0

ciclo: la \$a0, car0 # stringa "0"

li \$v0, 4 # print string

syscall

j ciclo # ciclo infinito

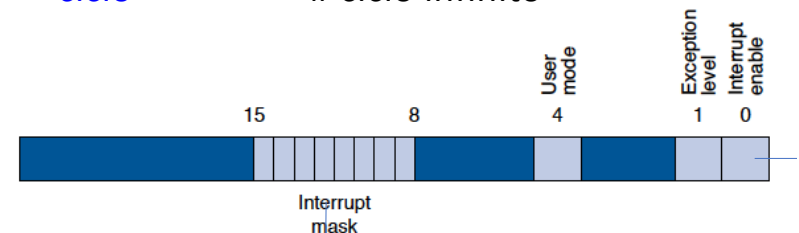


FIGURE A.7.1 The Status register.

Esempio (interrupt) – 2/3

Porzione di codice di myInterruptExceptionHandler.s per gestione interrupt da Receiver

IntKbd:

andi \$a0, \$k0, 0xFF00 #in k0 era stato copiato Cause

 # Interrupt mask: 0xFF00 = 1111 1111 0000 0000

srl \$a0, \$a0, 8

andi \$a0, \$a0, 8 # Receiver mask: 0000 1000 0000 0000

beq \$a0, \$0, EndIntKbd # branch se e' interrupt ma non di Receiver

E' interrupt del Receiver, quindi stampa le info sull'interrupt

li \$v0, 4

la \$a0, __m5_

syscall # print " by Receiver "

DISABILITA Interruzioni dal Receiver (tastiera)

lw \$a0, 0xFFFF0000 # Leggo attuale Receiver Control

andi \$a0, \$a0, 0xFFFD # Azzero bit 1 = INT enable

sw \$a0, 0xFFFF0000 # Scrivo nuovo Receiver Control

DISABILITA Interrupt dal Receiver nella CPU

mfc0 \$a0, \$12 # Leggo il CPU Status Reg.

andi \$a0, \$a0, 0xF7FF # Azzero il bit INT del Receiver

mtc0 \$a0, \$12 # Scrivo il nuovo CPU Status Reg.

Gestione del char digitato

lb \$a0, 0xFFFF0004 # prelevo il carattere dal Data Reg.

sb \$a0, Carattere # e lo metto in memoria

li \$v0, 4 # stampa con syscall

la \$a0, Messaggio # il carattere e' incorporato !

syscall

#(continua con EndIntKbd)

Esempio (interrupt) – 3/3

EndIntKbd:

ABILITA interrupt della CPU

```
mfc0 $t0, $12      # Leggi CPU Status register
ori  $t0, $t0, 1    # abilita in generale gli INTERRUPT (mettendo a 1 il ready bit, bit 0)
ori  $t0, $t0, 0x800 # abilita INTERRUPT dal Receiver (Receiver mask: 0x0800 = 0000 1000 0000 0000)
mtc0 $t0, $12      # Scrivi nuovo CPU Status register
```

ABILITA Interrupt dal Receiver (tastiera-keyboard)

```
lw  $t1, 0xFFFF0000 # Leggi attuale Receiver Control
ori $t1, $t1, 2      # abilita il bit 1 (bit di INTERRUPT enable)
sw  $t1, 0xFFFF0000 # scrivi nuovo Receiver Control
```

Esercizio 1

Determinare la percentuale di tempo in cui viene effettivamente utilizzata la CPU per trasferire **1 dato della dimensione di una word da una tastiera** utilizzando la tecnica di **gestione a controllo di programma** se:

- la tastiera trasferisce **10 byte/s**
- frequenza di clock: **250 MHz**
- sono richiesti **20 cicli di clock per ogni byte**

Esercizio 1 – soluzione

Numero di cicli in cui la CPU è occupata nel trasferimento

- Poichè sono richiesti 20 cicli per trasferire ogni byte $\rightarrow$ sono richiesti $20 \cdot 4 = 80$ cicli della CPU per trasferire 4 byte

Numero di cicli totali (effettuati nel tempo di trasferimento dalla periferica)

- Ma se il trasferimento della tastiera avviene 10 byte al sec $\rightarrow$ il tempo richiesto per il trasferimento di 4 byte è $4/10 = 0,4$ sec
- In 0,4 sec, con una frequenza di 250 MHz (250 milioni di cicli al secondo), sono effettuati $= 250 \cdot 10^6 * 0,4 = 100 \cdot 10^6$ cicli
- **% uso effettivo CPU** $= (80 / 10 \cdot 10^7) = 8 \cdot 10^{-7} = 0,00008 \%$

Esercizio 2

Qual è la frazione del tempo del processore usata per la **gestione degli interrupt** necessari al trasferimento da/verso una periferica di I/O, se abbiamo:

- Frequenza di clock di **250 Mhz**
- Trasferimento di **4 Mbyte/s**, a blocchi di **4 word**
- Sono richiesti **500 cicli di clock** per ogni interruzione

Esercizio 2 – soluzione

Numero di cicli in cui la CPU è occupata nel trasferimento

- Poiché il trasferimento è di 4 Mbyte al secondo e poichè abbiamo un interrupt ogni volta che sono trasferiti 16 byte (il trasferimento è a blocchi di 4 word) abbiamo

$$4 \cdot 10^6 \text{ byte/s} / 16 \text{ byte} = \mathbf{250.000 \text{ interrupt/s}}$$

- Sono richiesti 500 cicli per gestire ogni interrupt e quindi per la gestione delle interruzioni sono richiesti in totale:

$$250.000 \text{ interrupt/s} * 500 \text{ cicli/interrupt} = \mathbf{125.000.000 \text{ cicli/sec}}$$

Frequenza del clock: 250.000.000 cicli al secondo

- Frazione di utilizzo del processore per la gestione degli interrupt: $125M/250M = \mathbf{50\%}$

Esercizio 3

Qual è la frazione del tempo del processore usata per la **gestione con DMA** di un trasferimento dati da periferica, se abbiamo:

- Frequenza di clock è **250 Mhz**
- Trasferimento a blocchi di **8 kbyte per ogni DMA**
- Trasferimento a **4 Mbyte/s**
- Il costo dell'inizializzazione del DMA è di **1000 cicli di clock**
- Il costo dell'interruzione al termine del DMA è di **500 cicli di clock**

Esercizio 3 – soluzione

- Per ciascun trasferimento DMA servono: **(1000 + 500) cicli di clock**

Numero di DMA/sec

- Poiché il trasferimento avviene alla velocità di 4 Mbyte/s e a blocchi di 8 kbyte abbiamo

$$4 \text{ Mbyte/s} / 8 \text{ kbyte} = \mathbf{500 \text{ DMA/sec}}$$

Numero cicli_clock CPU richiesti: $1500 * 500 = \mathbf{750.000 \text{ cicli/sec}}$

Frazione del processore utilizzata per DMA: $750k / 250M = \mathbf{0,3 \%}$

Esercizio 4

Si supponga di voler trasferire un testo di 2 Mbyte e che

1. il clock della CPU sia di 500 MHz ($500 \cdot 10^6$ Hz)
2. il device emetta al massimo 1 KB/s (1000 byte/s)
3. impostare i registri per DMA richiede 10 cicli di clock
4. eseguire alcune istruzioni di prologo e epilogo per quanto riguarda il controllo di programma sia trascurabile

Qual è il **rapporto** tra il tempo di impiego della CPU **nel caso di DMA** e **di I/O a controllo di programma**?

Esercizio 4 (soluzione 1/3)

Si supponga di voler trasferire un testo di 2 Mbyte e che

1. il clock della CPU sia di 500 MHz ($500 \cdot 10^6$ Hz)
2. il device emetta al massimo 1 KB/s (1000 byte/s)
3. impostare i registri per DMA richiede 10 cicli di clock
4. eseguire alcune istruzioni di prologo e epilogo per quanto riguarda il controllo di programma sia trascurabile

Qual è il **rapporto** tra il tempo di impiego della CPU **nel caso di DMA** e **di I/O a controllo di programma**?

Tempo di impiego della CPU nel caso di DMA?

Cicli di clock della CPU = 10 (3)

Tempo CPU = $\frac{\text{num cicli clock CPU}}{\text{freq. clock}}$

$$= \frac{10}{500 \cdot 10^6} = 2 \cdot 10^{-8} \text{ sec}$$

Esercizio 4 (soluzione 2/3)

Si supponga di voler trasferire un testo di **2 Mbyte (0)** e che

1. il clock della CPU sia di 500 MHz ($500 \cdot 10^6$ Hz)
2. il device emetta al massimo 1 KB/s (1000 byte/s)
3. impostare i registri per DMA richiede 10 cicli di clock
4. eseguire alcune istruzioni di prologo e epilogo per quanto riguarda il controllo di programma sia trascurabile

Qual è il **rapporto** tra il tempo di impiego della CPU **nel caso di DMA** e **di I/O a controllo di programma**?

Tempo di impiego della CPU nel caso di I/O a controllo di programma?

Tempo CPU per esecuz. progr. dato da

$$\text{per (4)} \quad \frac{\text{Num. byte da trasmettere}}{\text{Velocità trasmissione}} =$$
$$\text{per (0, 2)} \quad = \frac{2 \cdot 10^6 \text{ byte}}{1 \cdot 10^3 \text{ byte/sec}} = 2 \cdot 10^3 \text{ sec}$$

Cicli di clock della CPU =

$$\begin{aligned} &= \text{Tempo CPU} \cdot \text{frequenza_clock} \\ \text{per (1)} \quad &= 2 \cdot 10^3 \cdot 500 \cdot 10^6 \\ &= 1000 \cdot 10^9 \\ &= 10^{12} \end{aligned}$$

Esercizio 4 (soluzione 3/3)

Si supponga di voler trasferire un testo di 2 Mbyte e che

1. la CPU esegua 1 istruzione ogni ciclo di clock
2. il clock della CPU sia di 500 MHz ($500 \cdot 10^6$ Hz)
3. il device emetta al massimo 1 KB/s (1000 byte/s)
4. impostare i registri per DMA richiede 10 cicli di clock
5. eseguire alcune istruzioni di prologo e epilogo per quanto riguarda il controllo di programma sia trascurabile

Qual è il **rapporto** tra il tempo di impiego della CPU nel caso di DMA e di I/O a controllo di programma?

Il rapporto tra num.cicli →

$$\frac{\text{cicli CPU per DMA}}{\text{cicli CPU programmed I/O}} = \frac{10}{10^{12}} = 10^{-11}$$

oppure rapporto tempo CPU

$$\frac{\text{Tempo CPU DMA}}{\text{Tempo CPU prog. I/O}} = \frac{2 \cdot 10^{-8}}{2 \cdot 10^3}$$

Esercizio 5

Determinare la percentuale di tempo in cui verrebbe effettivamente utilizzata la CPU per trasferire **1 parola** con gestione di programma da un floppy disk e da un hard disk, tenendo conto che sono richiesti **20 cicli di clock per ogni byte**, che la **frequenza di clock della CPU è 50 Mhz** e

1. Floppy disk opera a **50 Kbyte/s**
2. Hard-disk lavora a **2 Mbyte/s**

Esercizio 5 – soluzione

Cicli di CPU richiesti per trasferire la word: $20 * 4 = 80$ cicli di clock

1. Floppy-disk:

tempo di trasferimento: $4 \text{ byte} / 50 \text{ kbyte/s} = 0,08 * 10^{-3} \text{ s}$

$50 * 10^6 \text{ hz} \times 0,08 * 10^{-3} = 4 \times 10^3 \text{ cicli_clock} \quad \rightarrow 80/4.000 = \mathbf{2\%}$

2. Hard-disk:

tempo di trasferimento: $4 \text{ byte} / 2 \text{ Mbyte /s} = 2 * 10^{-6} \text{ s}$

$50 * 10^6 \text{ hz} \times 2 * 10^{-6} = 100 \text{ cicli_clock} \quad \rightarrow 80/100 = \mathbf{80\%}$

- Ulteriori esercizi di trovanoo:
 - Quiz I/O
 - Laboratorio 8 – Gestione I/O